Engaging Julia Software vscode

Julia

Julia is a high-level, high-performance programming language designed for technical and numerical computing, known for its speed and ease of use. Because of its popularity, we have pre-installed versions of Julia on each of our clusters. You can begin using Julia right away by running `module load julia` or by specifying the module specifically:

```
module load julia/1.10.1
```

Installing Packages

Julia organizes packages by the version of Julia you're running. When you install packages, a `.julia` folder automatically gets created in your home directory that holds all package installations. Julia will automatically create an environment for that version, which will be saved in `~/.julia/environments`.

You can change the default package install location by setting the `$JULIA_DEPOT_PATH` environment variable from the command line before you start Julia. For example:

```
export JULIA_DEPOT_PATH=/home/$USER/orcd/r8/pool
```

Juliaup

Juliaup provides a convenient way to manage different versions of Julia and different package installations. It can be installed by running the following command:

```
curl -fsSL https://install.julialang.org | sh
```

You will be asked if you want to proceed with default settings or to customize your installation. We recommend customizing your installation. The default settings are as follows:

1. Save the `.juliaup` folder to your home directory. This folder contains all installations of Julia and their associated packages that are managed by Juliaup.

2. Edit your `.bashrc` and `.bash_profile` files to automatically add the Juliaup-managed version of Julia to your `$PATH` environment variable.
    - We generally discourage editing your `.bashrc` file because it can cause issues when trying to use other software. For example, if you want to use a pre-installed Julia module, you would have to manually remove Juliaup from your `$PATH` any time you connect to the cluster.
    - To add Juliaup to your `$PATH` manually, run:

```
export PATH=/path/to/.juliaup/bin${PATH:+:${PATH}}
```

Click here for more information on installing and using Juliaup.

Using Different Julia Versions

Juliaup Versions

If you're using Juliaup, installing different versions of Julia is straightforward:

```
# Install Julia 1.9.0:
juliaup add 1.9.0
# Use Julia 1.9.0:
julia +1.9.0
```

Note that Juliaup installs versions and packages to your `.julia` folder.

Manual Installation

If you are unable to use Juliaup and you need a version of Julia that is not pre-installed on the cluster, you can manually download it.

A complete list of previous Julia versions can be found here. From this site, copy the link to the `.tar.gz` file that corresponds to the version you need. Be sure to select the version for a Linux operating system and x86_64 architecture.

Download the `.tar.gz` file:

```
wget [link to file]
```

Extract the `.tar.gz` file:

```
tar -xvzf [file name]
```

Add the downloaded version to your path:

```
export PATH="~/path/to/julia/bin:$PATH"
```

The following example is for Julia 1.9.0:

```
wget https://julialang-s3.julialang.org/bin/linux/x64/1.9/julia-1.9.0-linux-x86_64.tar.gz
tar -xvzf julia-1.9.0-linux-x86_64.tar.gz
export PATH="~/julia-1.9.0/bin:$PATH"
```

Jupyter Notebooks

While Jupyter is heavily integrated with Python, it supports compatibility with Julia. You can run Jupyter notebooks on OnDemand. You can create a Julia kernel using either one of the pre-installed Julia modules, or any other Julia version you've installed in your home directory.

To create a kernel, add the `IJulia` package and build it in your Julia environment:

```
using Pkg
Pkg.add("IJulia")
Pkg.build("IJulia")
```

This should create a Julia Kernel that you can select for your Jupyter notebooks

VS Code

Please refer to the VS Code page for using VS Code on the cluster.

VS Code supports compatibility with Jupyter notebooks. If you have installed and built `IJulia` in your Julia environment, then you should be able to find the correct Julia kernel by navigating to `Select Kernel` > `Select Another Kernel` > `Jupyter Kernel`.

FAQs

I have loaded/installed a specific version of Julia, but it is not being recognized. What do I do?

There may be another/no version of Julia in your `PATH` environment variable. You can check this by running `echo $PATH`.

If you have loaded a Julia module but do not want to use it, you can run `module purge`.

February 12, 2026